

Escuela Técnica N°1 Otto Krause



Laboratorio de Programación Orientada a Objetos

Guión - Ejercitación 15

**Alumnos: Toledano Jazmin
Pagni Octavio**

Curso: 5°2 Computación

Prof: Aaron Sebastian Serrano

Ciclo Lectivo 2026

Consignas:

1. Idea del juego en 3-4 líneas: de qué se trata, qué hace el jugador.

Decidimos utilizar la idea de nuestro proyecto cuatrimestral de proyecto informático, un juego ambientado en el buffet de la escuela.

En este juego, el jugador tiene que realizar diferentes tareas, atender a los clientes que llegan al buffet y administrar los objetos que posee en su inventario.

El juego se desarrollará completamente mediante líneas de texto plano en una consola, donde el jugador deberá seleccionar diferentes opciones para realizar las acciones disponibles.

ORDEN DE LAS CARPETAS Y SUS RESPECTIVAS CLASES:

MODEL

- |
- |— Entidad
- |— Personaje
- |— Jugador
- |— NPC
- |— Cliente
- |— Objeto
- |— Tarea
- |— Inventario

INTERFACES

- |
- |— Interactable

DAO

- |
- |— EntidadDAO
- |— Implementación correspondiente

SERVICE

- |
- |— EntidadService

EXCEPCIONES

- |
- |— InteraccionInvalidaException
- |— TareaNoDisponibleException
- |— ObjetoNoDisponibleException

```
├── ClienteNoDisponibleException
├── OpcionInvalidaException
```

MAIN

```
├── Main
```

2. Las clases del modelo: cuál es la clase base, cuáles heredan, y por qué eligieron esa jerarquía (qué comparten, qué las diferencia).

La clase base de nuestro modelo es **Entidad**, que será una clase abstracta. Esta clase tendrá los atributos que comparten los elementos principales del juego, como id y nombre.

Entidad heredan 3 subclases, que son las siguientes:

Personaje

↓

Objeto

↓

Tarea

Personaje hereda a Jugador y NPCs.

NPC hereda a Cliente.

Objeto no hereda a nada, ya que representa los objetos que pueden encontrar y utilizarse durante el juego.

Tarea tampoco hereda a nada, ya que representa las diferentes actividades que el jugador debe realizar.

Existe también la clase Inventario, que se encarga de almacenar los objetos que posee el jugador.

Elegimos esta jerarquía porque las clases principales representan diferentes elementos del juego y comparten características generales, pero independientemente tienen sus propias características y comportamientos. Por ejemplo, un Jugador tiene el día actual, dinero y tareas completadas, mientras que un Cliente tiene un pedido y un NPC puede tener un rol.

3. Atributos de cada clase (los del padre + los específicos de cada hija).

Atributos de las clases:

Entidad

- id
- nombre

Personaje

Además de los atributos heredados de Entidad:

- diálogo

Jugador

Además de los atributos heredados de Personaje:

- diaActual
- tareasCompletadas
- dinero

NPC

Además de los atributos heredados de Personaje:

- rol

Cliente

Además de los atributos heredados de NPC:

- pedido

Objeto

Además de los atributos heredados de Entidad:

- tipoObjeto
- utilizable
- cantidad

Tarea

La clase Tarea representará las actividades que el jugador debe completar durante los diferentes días.

- id
- descripción
- tipo
- día
- completada

Inventario

La clase Inventario permitirá almacenar los objetos que el jugador posee durante la partida.

- objetos
- cantidades

4. Diseño de la tabla en MySQL (el CREATE TABLE, igual que venimos haciendo, con la columna tipo si van a usar tabla única para las subclases).

Decidimos utilizar una única tabla, como venimos haciendo desde que empezamos este tema para que sea más simple. (Base de datos se llama Buffet)

```
CREATE TABLE entidades (  
id INT PRIMARY KEY AUTO_INCREMENT,  
nombre VARCHAR(50) NOT NULL,  
tipo VARCHAR(30) NOT NULL,  
dia_actual INT,  
tareas_completadas INT,  
dinero INT,  
dialogo VARCHAR(255),  
rol VARCHAR(50),  
pedido VARCHAR(100),  
tipo_objeto VARCHAR(50),  
utilizable BOOLEAN,  
cantidad INT,  
descripcion VARCHAR(255),  
tipo_tarea VARCHAR(50),  
completada BOOLEAN,  
dia INT  
);
```

El "tipo" sirve para saber si es un jugador, cliente, empleado, objeto o tarea.

5. Lista de reglas de negocio que va a validar el service (mínimo 5), y qué excepción dispara cada una.

- Si la tarea no está disponible: **TareaNoDisponibleException**
- Si el objeto no está disponible: **ObjetoNoDisponibleException**
- Si no hay ningún cliente disponible: **ClienteNoDisponibleException**
- Si se selecciona una opción que no existe: **OpcionInvalidaException**
- Si no se puede interactuar: **InteraccionInvalidaException**

Qué interfaz van a definir y en qué método del service la van a usar de forma polimórfica.

Interfaz que vamos a usar: Interactuable

Método en que la vamos a usar: void interactuar();

La interfaz va a ser usada en clases que puedan tener interacción. (Ej: el objeto, el cliente o la tarea)

7. Métodos previstos del service (nombres, no implementación)

Métodos para las entidades

crearEntidad()
buscarEntidad()
listarEntidades()
eliminarEntidad()

Métodos para la jugabilidad

realizarTarea()
atenderCliente()
esperarCliente()
abrirInventario()
usarObjeto()
avanzarDia()

Métodos para los clientes

generarCliente()
mostrarPedido()
entregarPedido()

Métodos para las tareas

generarTarea()
mostrarTarea()
completarTarea()

Métodos para el inventario

mostrarInventario()
agregarObjeto()
eliminarObjeto()

Método polimórfico

interactuar(Interactuable elemento)

El Service podrá recibir diferentes tipos de objetos interactivables y ejecutar su comportamiento correspondiente sin depender de una subclase específica.

Fin:D